



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



UNIVERSITÀ DEGLI STUDI DI PADOVA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Sviluppo firmware per la gestione del sistema di riscaldamento di un prototipo di incubatrice neonatale

Relatore

Prof. Rampazzo Mirco

Laureando

Simonetto Giacomo
2109923

ANNO ACCADEMICO 2025-2026

Data di laurea 22/09/2026

Abstract

Il seguente elaborato finale descrive lo sviluppo del firmware per la gestione del sistema di riscaldamento e delle varie periferiche del prototipo di incubatrice neonatale costruito da Vanni Bincoletto nella sua tesi magistrale in bioingegneria [1].

Si inizia trattando la progettazione e realizzazione su breadboard del circuito necessario per collegare le varie periferiche del prototipo alla scheda programmabile Arduino Mega 2560. Si prosegue analizzando il firmware sviluppato per il controllo delle periferiche, il log dei dati su computer e il sistema di gestione della temperatura tramite un semplice controllore ad isteresi e un più evoluto controllore PID. Si conclude con la taratura dei parametri del controllore PID e l'analisi dei risultati ottenuti, evidenziando i vantaggi e gli svantaggi dei due controllori implementati.

Indice

Abstract	iii
Elenco delle figure	vii
Elenco delle tabelle	ix
Elenco dei codici	xi
1 Sviluppo del firmware per la gestione dell'incubatrice neonatale	1
1.1 Requisiti del firmware	1
1.2 Configurazione generale	2
1.3 Gestione degli attuatori	2
1.4 Gestione del pannello frontale	2
1.5 Gestione della comunicazione seriale al PC	2
1.6 Gestione del sensore di temperatura e umidità	2
Bibliografia	3

Elenco delle figure

Elenco delle tabelle

Elenco dei codici

Capitolo 1

Sviluppo del firmware per la gestione dell'incubatrice neonatale

1.1 Requisiti del firmware

Il firmware per il microcontrollore deve essere in grado di gestire in tempo reale ed in maniera concorrente tutti i componenti del prototipo di incubatrice neonatale, come il pannello informativo frontale, la gestione degli attuatori, il sensore di temperatura e umidità, la comunicazione seriale con il PC e i controllori per la gestione della temperatura.

Per garantire la corretta sincronizzazione tra i vari processi, non è possibile impiegare codice bloccante come loop in attesa di soddisfare una certa condizione o l'utilizzo della funzione `delay()`. Serve usare un approccio basato su macchine a stati e condizioni che sfruttano la funzione `millis()`. In questo modo, il microcontrollore può portare avanti più processi in parallelo senza rimanere occupato a concludere un determinato compito e trascurare gli altri.

Siccome il firmware deve operare su un microcontrollore con risorse limitate, l'implementazione deve essere efficiente sia in termini di complessità computazionale che di memoria. Per questo motivo, si cerca di limitare il più possibile l'uso di variabili locali e di allocazioni dinamiche nello stack/heap, preferendo l'uso di variabili globali. In questo modo la memoria RAM utilizzata per le variabili è già conosciuta a tempo di compilazione e risulta facile accorgersi di possibili problemi di overflow della memoria. Inoltre, l'uso di variabili globali permette di condividere facilmente lo stato del sistema tra i vari moduli del firmware, senza dover ricorrere continuamente a passaggi di parametri tra le funzioni.

Infine è richiesto che il codice sia modulare, leggibile, documentato e facilmente modificabile. Nello sviluppo di un prototipo orientato alla ricerca, infatti, capiterà sicuramente e molto spesso la necessità di modificare il codice per aggiungere nuove funzionalità, correggere scelte che si rivelano non ottimali o per adattarlo a nuove esigenze.

1.2 Configurazione generale

- variabili globali di stato
- files e moduli del firmware
- istruzioni di compilazione da arduino ide e platform.io

1.3 Gestione degli attuatori

- controllo manuale o automatico degli attuatori
- gestione centralizzata degli attuatori per aggiornare le variabili di stato
- gestione della ventola di omogeneizzazione della temperatura
- gestione del refill dell'acqua per il sistema di umidificazione

1.4 Gestione del pannello frontale

- lettura degli switch, con impostazione controllo manuale
- lettura dei potenziometri, con filtri per ridurre il rumore
- gestione del display lcd con temporizzazioni
- gestione degli errori e gestione del led di allarme

1.5 Gestione della comunicazione seriale al PC

- comunicazioni seriali
- codice matlab

1.6 Gestione del sensore di temperatura e umidità

- gestione non bloccante
- utilizzo di una macchina a stati

Bibliografia

- [1] V. Bincoletto, «Realizzazione e modellazione di un prototipo di incubatrice neonatale,»
Tesi magistrale in Bioingegneria, Università di Padova, 2025.